O'REILLY®

Production Kubernetes

Building Successful Application Platforms

Josh Rosso, Rich Lander,
Alexander Brand & John Harris

O'REILLY®

Production Kubernetes

Although Kubernetes has become the dominant container orchestrator, many organizations relatively new to this system struggle to run actual production workloads. In this practical book, software engineers Josh Rosso, Rich Lander, Alexander Brand, and John Harris from VMware share their experiences running Kubernetes in production and provide insight on key challenges and best practices.

The brilliance of Kubernetes is how configurable and extensible it is, from pluggable runtimes to storage integrations. For platform engineers, software developers, infosec folks, network engineers, storage engineers, and others, this book examines how the path to success with Kubernetes involves a variety of technology, pattern, and abstraction considerations.

With this guide, you will:

- Understand the capabilities required to build a robust Kubernetes-based platform
- Learn from real-world experience to avoid the pitfalls that often occur when building a Kubernetes-based platform
- Discover how Kubernetes's architecture and design enables extensible platform development
- Analyze the concerns of internal and external users to develop a platform that's fit for purpose
- Control the complexity of your Kubernetes platform by making reasoned decisions about tooling and abstractions
- Examine the path to production with Kubernetes and learn about common tooling options and trade-offs

"So much good advice. I wish I'd had this book when designing clusters for the first time."

—**Michael Goodness**
Principal DevOps Engineer, MLB

"This book is a must-read if you are tasked with replatforming or evaluating the effort involved in adopting Kubernetes for infrastructure teams."

—**Duffie Cooley**
CNCF Ambassador

Josh Rosso is an engineer who's worked with Kubernetes at CoreOS (Red Hat), Heptio, and VMware.

Rich Lander is a VMware field engineer who helps enterprises adopt Kubernetes and cloud native technology.

Alexander Brand is a software engineer focused on Kubernetes and cloud native technologies.

John Harris is a staff engineer who's worked on cloud native tooling, platforms, and patterns at VMware (Heptio) and Docker.

KUBERNETES

US $69.99 CAN $92.99
ISBN: 978-1-492-09230-8

Twitter: @oreillymedia
facebook.com/oreilly

Production Kubernetes

Building Successful Application Platforms

Josh Rosso, Rich Lander,
Alexander Brand, and John Harris

Beijing • Boston • Farnham • Sebastopol • Tokyo

O'REILLY®

Production Kubernetes
by Josh Rosso, Rich Lander, Alexander Brand, and John Harris

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (*http://oreilly.com*). For more information, contact our corporate/institutional sales department: 800-998-9938 or *corporate@oreilly.com*.

Acquisitions Editor: John Devins
Development Editor: Jeff Bleiel
Production Editor: Christopher Faucher
Copyeditor: Kim Cofer
Proofreader: Piper Editorial Consulting, LLC

Indexer: Ellen Troutman
Interior Designer: David Futato
Cover Designer: Karen Montgomery
Illustrator: Kate Dullea

March 2021: First Edition

Revision History for the First Edition
2021-03-16: First Release

See *http://oreilly.com/catalog/errata.csp?isbn=9781492092308* for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *Production Kubernetes*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

This work is part of a collaboration between O'Reilly and VMware Tanzu. See our statement of editorial independence.

978-1-492-09230-8

[LSI]

Table of Contents

Foreword

It has been more than six years since we publicly released Kubernetes. I was there at the start and actually submitted the first commit to the Kubernetes project. (That isn't as impressive as it sounds! It was a maintenance task as part of creating a clean repo for public release.) I can confidently say that the success Kubernetes has seen is something we had hoped for but didn't really expect. That success is based on a large community of dedicated and welcoming contributors along with a set of practitioners who bridge the gap to the real world.

I'm lucky enough to have worked with the authors of *Production Kubernetes* at the startup (Heptio) that I cofounded with the mission to bring Kubernetes to typical enterprises. The success of Heptio is, in large part, due to my colleagues' efforts in creating a direct connection with real users of Kubernetes who are solving real problems. I'm grateful to each one of them. This book captures that on-the-ground experience to give teams the tools they need to really make Kubernetes work in a production environment.

My entire professional career has been based on building systems aimed at application teams and developers. It started with Microsoft Internet Explorer and then continued with Windows Presentation Foundation and then moved to cloud with Google Compute Engine and Kubernetes. Again and again I've seen those building platforms suffer from what I call "The Platform Builder's Curse." The people who are building the platforms are focused on a longer time horizon and the challenge of building a foundation that will, hopefully, last decades. But that focus creates a blind spot to the problems that users are having *right now*. Oftentimes we are so busy building a thing we don't have the time and problems that lead us to actually use the thing we are building.

The only way to defeat the platform builder's curse is to actively seek information from outside our platform-builder bubble. This is what the Heptio Field Engineering team (and later the VMware Kubernetes Architecture Team—KAT) did for me. Beyond helping a wide variety of customers across industries be successful with Kubernetes, the team is a critical window into the reality of how the "theory" of our platform is applied.

This problem is only exacerbated by the thriving ecosystem that has been built up around Kubernetes and the Cloud Native Computing Foundation (CNCF). This includes both projects that are part of the CNCF and those that are in the larger orbit. I describe this ecosystem as "beautiful chaos." It is a rainforest of projects with varying degrees of overlap and maturity. This is what innovation looks like! But, just like exploring a rainforest, exploring this ecosystem requires dedication and time, and it comes with risks. New users to the world of Kubernetes often don't have the time or capacity to become experts in the larger ecosystem.

Production Kubernetes maps out the parts of that ecosystem, when individual tools and projects are appropriate, and demonstrates how to evaluate the right tool for the problems the reader is facing. This advice goes beyond just telling readers to use a particular tool. It is a larger framework for understanding the problem a class of tools solves, knowing whether you have that problem, being familiar with the strengths and weaknesses to different approaches, and offering practical advice for getting going. For those looking to take Kubernetes into production, this information is gold!

In conclusion, I'd like to send a big "Thank You" to Josh, Rich, Alex, and John. Their experience has made many customers directly successful, has taught me a lot about the thing that we started more than six years ago, and now, through this book, will provide critical advice to countless more users.

— Joe Beda
Principal Engineer for VMware Tanzu,
Cocreator of Kubernetes,
Seattle, January 2021

Preface

Kubernetes is a remarkably powerful technology and has achieved a meteoric rise in popularity. It has formed the basis for genuine advances in the way we manage software deployments. API-driven software and distributed systems were well established, if not widely adopted, when Kubernetes emerged. It delivered excellent renditions of these principles, which are foundational to its success, but it also delivered something else that is vital. In the recent past, software that autonomously converged on declared, desired state was possible only in giant technology companies with the most talented engineering teams. Now, highly available, self-healing, autoscaling software deployments are within reach of every organization, thanks to the Kubernetes project. There is a future in front of us where software systems accept broad, high-level directives from us and execute upon them to deliver desired outcomes by discovering conditions, navigating changing obstacles, and repairing problems without our intervention. Furthermore, these systems will do it faster and more reliably than we ever could with manual operations. Kubernetes has brought us all much closer to that future. However, that power and capability comes at the cost of some additional complexity. The desire to share our experiences helping others navigate that complexity is why we decided to write this book.

You should read this book if you want to use Kubernetes to build a production-grade application platform. If you are looking for a book to help you get started with Kubernetes, or a text on how Kubernetes works, this is not the right book. There is a wealth of information on these subjects in other books, in the official documentation, and in countless blog posts and the source code itself. We recommend pairing the consumption of this book with your own research and testing for the solutions we discuss, so we rarely dive deeply into *step-by-step* tutorial style examples. We try to cover as much theory as necessary and leave most of the *implementation* as an exercise to the reader.

Throughout this book, you'll find guidance in the form of options, tooling, patterns, and practices. It's important to read this guidance with an understanding of how the authors view the practice of building application platforms. We are engineers and architects who get deployed across many Fortune 500 companies to help them take their platform aspirations from idea to production. We have been using Kubernetes as the foundation for getting there since as early as 2015, when Kubernetes reached `1.0`. We have tried as much as possible to focus on patterns and philosophy rather than on tools, as new tooling appears quicker than we can write! However, we inevitably have to demonstrate those patterns with the most appropriate tool du jour.

We have had major successes guiding teams through their cloud native journey to completely transform how they build and deliver software. That said, we have also had our doses of failure. A common reason for failure is an organization's misconception of what Kubernetes will solve for. This is why we dive so deep into the concept early on. Over this time we've found several areas to be especially interesting for our customers. Conversations that help customers get further on their path to production, or even help them define it, have become routine. These conversations became so common that we decided maybe it's time to write a book!

While we've made this journey to production with organizations time and time again, there is only one key consistency across them. This is that the road *never* looks the same, no matter how badly we sometimes want it to. With this in mind, we want to set the expectation that if you're going into this book looking for the "5-step program" for getting to production or the "10 things every Kubernetes user should know," you're going to be frustrated. We're here to talk about the many decision points and the traps we've seen, and to back it up with concrete examples and anecdotes when appropriate. Best practices exist but must always be viewed through the lens of pragmatism. There is no one-size-fits-all approach, and "It depends" is an entirely valid answer to many of the questions you'll inevitably confront on the journey.

That said, we highly encourage you to *challenge* this book! When working with clients we're always encouraging them to challenge and augment our guidance. Knowledge is fluid, and we are always updating our approaches based on new features, information, and constraints. You should continue that trend; as the cloud native space continues to evolve, you'll certainly decide to take alternative roads from what we recommended. We're here to tell you about the ones we've been down so you can weigh our perspective against your own.

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic
: Indicates new terms, URLs, email addresses, filenames, and file extensions.

`Constant width`
: Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords.

`Constant width bold`
: Shows commands or other text that should be typed literally by the user.

`Constant width italic`
: Shows text that should be replaced with user-supplied values or by values determined by context.

Kubernetes kinds are capitalized, as in Pod, Service, and StatefulSet.

This element signifies a tip or suggestion.

This element signifies a general note.

This element indicates a warning or caution.

Using Code Examples

Supplemental material (code examples, exercises, etc.) is available for download and discussion at *https://github.com/production-kubernetes*.

If you have a technical question or a problem using the code examples, please send email to *bookquestions@oreilly.com*.

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing examples from O'Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product's documentation does require permission.

We appreciate, but generally do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: "*Production Kubernetes* by Josh Rosso, Rich Lander, Alexander Brand, and John Harris (O'Reilly). Copyright 2021 Josh Rosso, Rich Lander, Alexander Brand, and John Harris, 978-1-492-09231-5."

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at *permissions@oreilly.com*.

O'Reilly Online Learning

For more than 40 years, *O'Reilly Media* has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O'Reilly's online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O'Reilly and 200+ other publishers. For more information, visit *http://oreilly.com*.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O'Reilly Media, Inc.
1005 Gravenstein Highway North
Sebastopol, CA 95472
800-998-9938 (in the United States or Canada)
707-829-0515 (international or local)
707-829-0104 (fax)

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at *https://oreil.ly/production-kubernetes*.

Email *bookquestions@oreilly.com* to comment or ask technical questions about this book.

For news and information about our books and courses, visit *http://oreilly.com*.

Find us on Facebook: *http://facebook.com/oreilly*

Follow us on Twitter: *http://twitter.com/oreillymedia*

Watch us on YouTube: *http://youtube.com/oreillymedia*

Acknowledgments

The authors would like to thank Katie Gamanji, Michael Goodness, Jim Weber, Jed Salazar, Tony Scully, Monica Rodriguez, Kris Dockery, Ralph Bankston, Steve Sloka, Aaron Miller, Tunde Olu-Isa, Alex Withrow, Scott Lowe, Ryan Chapple, and Kenan Dervisevic for their reviews and feedback on the manuscript. Thanks to Paul Lundin for encouraging the development of this book and for building the incredible Field Engineering team at Heptio. Everyone on the team has contributed in some way by collaborating on and developing many of the ideas and experiences we cover over the next 450 pages. Thanks also to Joe Beda, Scott Buchanan, Danielle Burrow, and Tim Coventry-Cox at VMware for their support as we initiated and developed this project. Finally, thanks to John Devins, Jeff Bleiel, and Christopher Faucher at O'Reilly for their ongoing support and feedback.

The authors would also like to personally thank the following people:

Josh: I would like to thank Jessica Appelbaum for her absurd levels of support, specifically blueberry pancakes, while I dedicated my time to this book. I'd also like to thank my mom, Angela, and dad, Joe, for being my foundation growing up.

Rich: I would like to thank my wife, Taylor, and children, Raina, Jasmine, Max, and John, for their support and understanding while I took time to work on this book. I would also like to thank my Mum, Jenny, and my Dad, Norm, for being great role models.

Alexander: My love and thanks to my amazing wife, Anais, who was incredibly supportive as I dedicated time to writing this book. I also thank my family, friends, and colleagues who have helped me become who I am today.

John: I'd like to thank my beautiful wife, Christina, for her love and patience during my work on this book. Also thanks to my close friends and family for their ongoing support and encouragement over the years.

CHAPTER 1

A Path to Production

Over the years, the world has experienced wide adoption of Kubernetes within organizations. Its popularity has unquestionably been accelerated by the proliferation of containerized workloads and microservices. As operations, infrastructure, and development teams arrive at this inflection point of needing to build, run, and support these workloads, several are turning to Kubernetes as part of the solution. Kubernetes is a fairly young project relative to other, massive, open source projects such as Linux. Evidenced by many of the clients we work with, it is still early days for most users of Kubernetes. While many organizations have an existing Kubernetes footprint, there are far fewer that have reached production and even less operating at scale. In this chapter, we are going to set the stage for the journey many engineering teams are on with Kubernetes. Specifically, we are going to chart out some key considerations we look at when defining a path to production.

Defining Kubernetes

Is Kubernetes a platform? Infrastructure? An application? There is no shortage of thought leaders who can provide you their precise definition of what Kubernetes is. Instead of adding to this pile of opinions, let's put our energy into clarifying the problems Kubernetes solves. Once defined, we will explore how to build atop this feature set in a way that moves us toward production outcomes. The ideal state of "Production Kubernetes" implies that we have reached a state where workloads are successfully serving production traffic.

The name *Kubernetes* can be a bit of an umbrella term. A quick browse on GitHub reveals the `kubernetes` organization contains (at the time of this writing) 69 repositories. Then there is `kubernetes-sigs`, which holds around 107 projects. And don't get us started on the hundreds of Cloud Native Compute Foundation (CNCF) projects that play in this landscape! For the sake of this book, *Kubernetes* will refer exclusively

to the core project. So, what is the core? The core project is contained in the kubernetes/kubernetes repository. This is the location for the key components we find in most Kubernetes clusters. When running a cluster with these components, we can expect the following functionality:

- Scheduling workloads across many hosts
- Exposing a declarative, extensible, API for interacting with the system
- Providing a CLI, `kubectl`, for humans to interact with the API server
- Reconciliation from current state of objects to desired state
- Providing a basic service abstraction to aid in routing requests to and from workloads
- Exposing multiple interfaces to support pluggable networking, storage, and more

These capabilities create what the project itself claims to be, a *production-grade container orchestrator*. In simpler terms, Kubernetes provides a way for us to run and schedule containerized workloads on multiple hosts. Keep this primary capability in mind as we dive deeper. Over time, we hope to prove how this capability, while foundational, is only part of our journey to production.

The Core Components

What are the components that provide the functionality we have covered? As we have mentioned, core components reside in the `kubernetes/kubernetes` repository. Many of us consume these components in different ways. For example, those running managed services such as Google Kubernetes Engine (GKE) are likely to find each component present on hosts. Others may be downloading binaries from repositories or getting signed versions from a vendor. Regardless, anyone can download a Kubernetes release from the `kubernetes/kubernetes` repository. After downloading and unpacking a release, binaries may be retrieved using the `cluster/get-kube-binaries.sh` command. This will auto-detect your target architecture and download server and client components. Let's take a look at this in the following code, and then explore the key components:

```
$ ./cluster/get-kube-binaries.sh

Kubernetes release: v1.18.6
Server: linux/amd64  (to override, set KUBERNETES_SERVER_ARCH)
Client: linux/amd64  (autodetected)

Will download kubernetes-server-linux-amd64.tar.gz from https://dl.k8s.io/v1.18.6
Will download and extract kubernetes-client-linux-amd64.tar.gz
Is this ok? [Y]/n
```

Inside the downloaded server components, likely saved to *server/kubernetes-server-${ARCH}.tar.gz*, you'll find the key items that compose a Kubernetes cluster:

API Server
: The primary interaction point for all Kubernetes components and users. This is where we get, add, delete, and mutate objects. The API server delegates state to a backend, which is most commonly etcd.

kubelet
: The on-host agent that communicates with the API server to report the status of a node and understand what workloads should be scheduled on it. It communicates with the host's container runtime, such as Docker, to ensure workloads scheduled for the node are started and healthy.

Controller Manager
: A set of controllers, bundled in a single binary, that handle reconciliation of many core objects in Kubernetes. When desired state is declared, e.g., three replicas in a Deployment, a controller within handles the creation of new Pods to satisfy this state.

Scheduler
: Determines where workloads should run based on what it thinks is the optimal node. It uses filtering and scoring to make this decision.

Kube Proxy
: Implements Kubernetes services providing virtual IPs that can route to backend Pods. This is accomplished using a packet filtering mechanism on a host such as `iptables` or `ipvs`.

While not an exhaustive list, these are the primary components that make up the core functionality we have discussed. Architecturally, Figure 1-1 shows how these components play together.

Kubernetes architectures have many variations. For example, many clusters run kube-apiserver, kube-scheduler, and kube-controller-manager as containers. This means the control-plane may also run a container-runtime, kubelet, and kube-proxy. These kinds of deployment considerations will be covered in the next chapter.

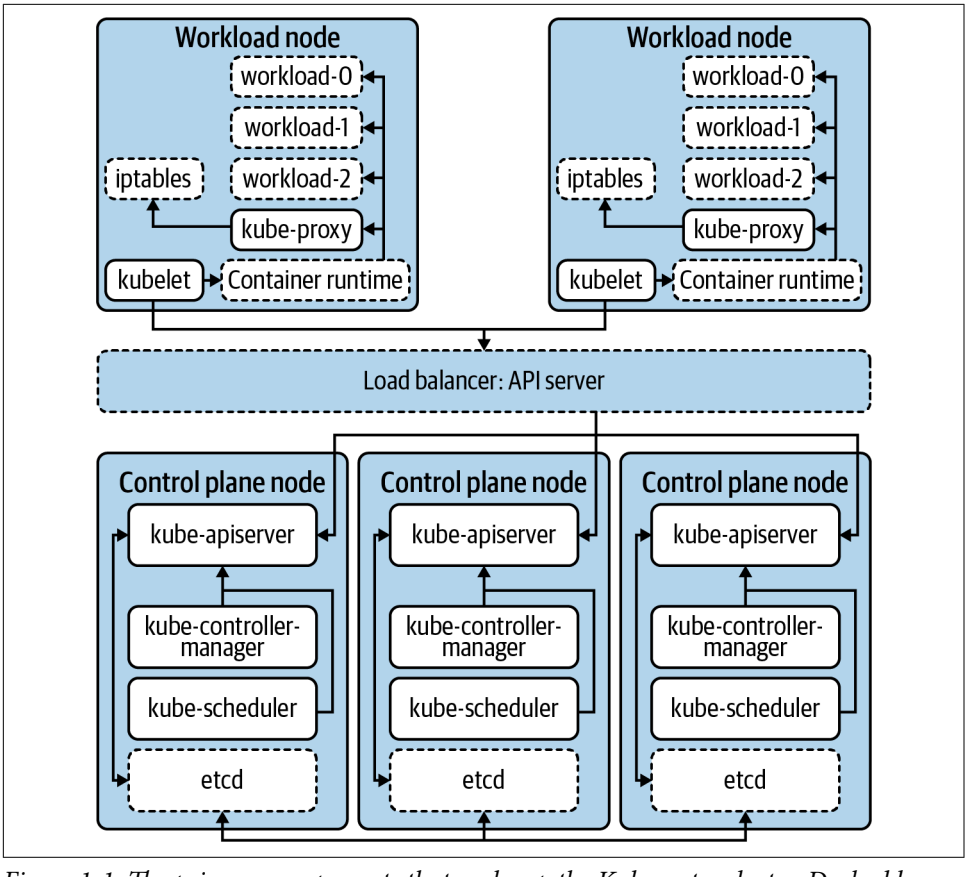

Figure 1-1. The primary components that make up the Kubernetes cluster. Dashed borders represent components that are not part of core Kubernetes.

Beyond Orchestration—Extended Functionality

There are areas where Kubernetes does more than just orchestrate workloads. As mentioned, the component kube-proxy programs hosts to provide a virtual IP (VIP) experience for workloads. As a result, internal IP addresses are established and route to one or many underlying Pods. This concern certainly goes beyond running and scheduling containerized workloads. In theory, rather than implementing this as part of core Kubernetes, the project could have defined a Service API and required a plug-in to implement the Service abstraction. This approach would require users to choose between a variety of plug-ins in the ecosystem rather than including it as core functionality.

This is the model many Kubernetes APIs, such as Ingress and NetworkPolicy, take. For example, creation of an Ingress object in a Kubernetes cluster does not guarantee

action is taken. In other words, while the API exists, it is not core functionality. Teams must consider what technology they'd like to plug in to implement this API. For Ingress, many use a controller such as ingress-nginx, which runs in the cluster. It implements the API by reading Ingress objects and creating NGINX configurations for NGINX instances pointed at Pods. However, `ingress-nginx` is one of many options. Project Contour implements the same Ingress API but instead programs instances of envoy, the proxy that underlies Contour. Thanks to this pluggable model, there are a variety of options available to teams.

Kubernetes Interfaces

Expanding on this idea of adding functionality, we should now explore interfaces. Kubernetes interfaces enable us to customize and build on the core functionality. We consider an interface to be a definition or contract on how something can be interacted with. In software development, this parallels the idea of defining functionality, which classes or structs may implement. In systems like Kubernetes, we deploy plug-ins that satisfy these interfaces, providing functionality such as networking.

A specific example of this interface/plug-in relationship is the Container Runtime Interface (CRI). In the early days of Kubernetes, there was a single container runtime supported, Docker. While Docker is still present in many clusters today, there is growing interest in using alternatives such as containerd or CRI-O. Figure 1-2 demonstrates this relationship with these two container runtimes.

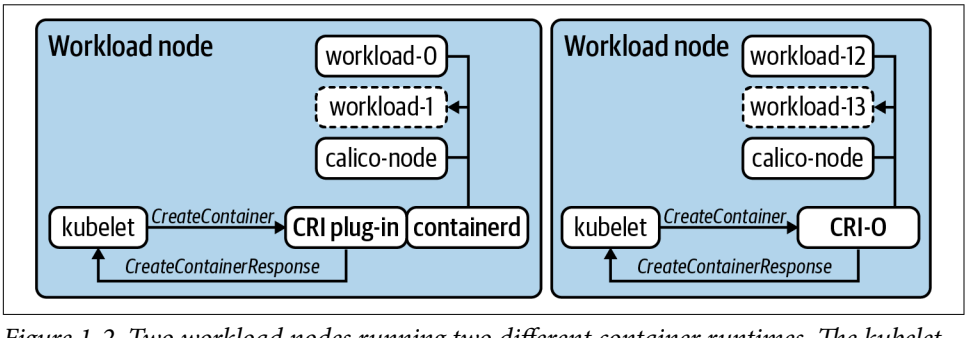

Figure 1-2. Two workload nodes running two different container runtimes. The kubelet sends commands defined in the CRI such as `CreateContainer` and expects the runtime to satisfy the request and respond.

In many interfaces, commands, such as `CreateContainerRequest` or `PortForwardRequest`, are issued as remote procedure calls (RPCs). In the case of CRI, the communication happens over GRPC and the kubelet expects responses such as `CreateContainerResponse` and `PortForwardResponse`. In Figure 1-2, you'll also notice two different models for satisfying CRI. CRI-O was built from the ground up as an implementation of CRI. Thus the kubelet issues these commands directly to it.

containerd supports a plug-in that acts as a shim between the kubelet and its own interfaces. Regardless of the exact architecture, the key is getting the container runtime to execute, without the kubelet needing to have operational knowledge of how this occurs for *every possible* runtime. This concept is what makes interfaces so powerful in how we architect, build, and deploy Kubernetes clusters.

Over time, we've even seen some functionality removed from the core project in favor of this plug-in model. These are things that historically existed "in-tree," meaning within the `kubernetes/kubernetes` code base. An example of this is cloud-provider integrations (CPIs). Most CPIs were traditionally baked into components such as the kube-controller-manager and the kubelet. These integrations typically handled concerns such as provisioning load balancers or exposing cloud provider metadata. Sometimes, especially prior to the creation of the Container Storage Interface (CSI), these providers provisioned block storage and made it available to the workloads running in Kubernetes. That's a lot of functionality to live in Kubernetes, not to mention it needs to be re-implemented for every possible provider! As a better solution, support was moved into its own interface model, e.g., kubernetes/cloud-provider, that can be implemented by multiple projects or vendors. Along with minimizing sprawl in the Kubernetes code base, this enables CPI functionality to be managed out of band of the core Kubernetes clusters. This includes common procedures such as upgrades or patching vulnerabilities.

Today, there are several interfaces that enable customization and additional functionality in Kubernetes. What follows is a high-level list, which we'll expand on throughout chapters in this book:

- The Container Networking Interface (CNI) enables networking providers to define how they do things from IPAM to actual packet routing.
- The Container Storage Interface (CSI) enables storage providers to satisfy intracluster workload requests. Commonly implemented for technologies such as ceph, vSAN, and EBS.
- The Container Runtime Interface (CRI) enables a variety of runtimes, common ones including Docker, containerd, and CRI-O. It also has enabled a proliferation of less traditional runtimes, such as firecracker, which leverages KVM to provision a minimal VM.
- The Service Mesh Interface (SMI) is one of the newer interfaces to hit the Kubernetes ecosystem. It hopes to drive consistency when defining things such as traffic policy, telemetry, and management.
- The Cloud Provider Interface (CPI) enables providers such as VMware, AWS, Azure, and more to write integration points for their cloud services with Kubernetes clusters.

- The Open Container Initiative Runtime Spec. (OCI) standardizes image formats ensuring that a container image built from one tool, when compliant, can be run in any OCI-compliant container runtime. This is not directly tied to Kubernetes but has been an ancillary help in driving the desire to have pluggable container runtimes (CRI).

Summarizing Kubernetes

Now we have focused in on the scope of Kubernetes. It is a *container orchestrator*, with a couple extra features here and there. It also has the ability to be extended and customized by leveraging plug-ins to interfaces. Kubernetes can be foundational for many organizations looking for an elegant means of running their applications. However, let's take a step back for a moment. If we were to take the current systems used to run applications in your organization and replace them with Kubernetes, would that be enough? For many of us, there is much more involved in the components and machinery that make up our current "application platform."

Historically, we have witnessed a lot of pain when organizations hold the view of having a "Kubernetes" strategy—or when they assume that Kubernetes will be an adequate forcing function for modernizing how they build and run software. Kubernetes is a technology, a great one, but it really should not be the focal point of where you're headed in the modern infrastructure, platform, and/or software realm. We apologize if this seems obvious, but you'd be surprised how many executive or higher-level architects we talk to who believe that Kubernetes, by itself, is the answer to problems, when in actuality their problems revolve around application delivery, software development, or organizational/people issues. Kubernetes is best thought of as a piece of your puzzle, one that enables you to deliver platforms for your applications. We have been dancing around this idea of an application platform, which we'll explore next.

Defining Application Platforms

In our path to production, it is key that we consider the idea of an application platform. We define an application platform as a viable place to run workloads. Like most definitions in this book, how that's satisfied will vary from organization to organization. Targeted outcomes will be vast and desirable to different parts of the business—for example, happy developers, reduction of operational costs, and quicker feedback loops in delivering software are a few. The application platform is often where we find ourselves at the intersection of apps and infrastructure. Concerns such as developer experience (devx) are typically a key tenet in this area.

Application platforms come in many shapes and sizes. Some largely abstract underlying concerns such as the IaaS (e.g., AWS) or orchestrator (e.g., Kubernetes). Heroku is a great example of this model. With it you can easily take a project written in languages like Java, PHP, or Go and, using one command, deploy them to production.

Alongside your app runs many platform services you'd otherwise need to operate yourself. Things like metrics collection, data services, and continuous delivery (CD). It also gives you primitives to run highly available workloads that can easily scale. Does Heroku use Kubernetes? Does it run its own datacenters or run atop AWS? Who cares? For Heroku users, these details aren't important. What's important is delegating these concerns to a provider or platform that enables developers to spend more time solving business problems. This approach is not unique to cloud services. RedHat's OpenShift follows a similar model, where Kubernetes is more of an implementation detail and developers and platform operators interact with a set of abstractions on top.

Why not stop here? If platforms like Cloud Foundry, OpenShift, and Heroku have solved these problems for us, why bother with Kubernetes? A major trade-off to many prebuilt application platforms is the need to conform to their view of the world. Delegating ownership of the underlying system takes a significant operational weight off your shoulders. At the same time, if how the platform approaches concerns like service discovery or secret management does not satisfy your organizational requirements, you may not have the control required to work around that issue. Additionally, there is the notion of vendor or opinion lock-in. With abstractions come opinions on how your applications should be architected, packaged, and deployed. This means that moving to another system may not be trivial. For example, it's significantly easier to move workloads between Google Kubernetes Engine (GKE) and Amazon Elastic Kubernetes Engine (EKS) than it is between EKS and Cloud Foundry.

The Spectrum of Approaches

At this point, it is clear there are several approaches to establishing a successful application platform. Let's make some big assumptions for the sake of demonstration and evaluate theoretical trade-offs between approaches. For the average company we work with, say a mid to large enterprise, Figure 1-3 shows an arbitrary evaluation of approaches.

In the bottom-left quadrant, we see deploying Kubernetes clusters themselves, which has a relatively low engineering effort involved, especially when managed services such as EKS are handling the control plane for you. These are lower on production readiness because most organizations will find that more work needs to be done on top of Kubernetes. However, there are use cases, such as teams that use dedicated cluster(s) for their workloads, that may suffice with just Kubernetes.

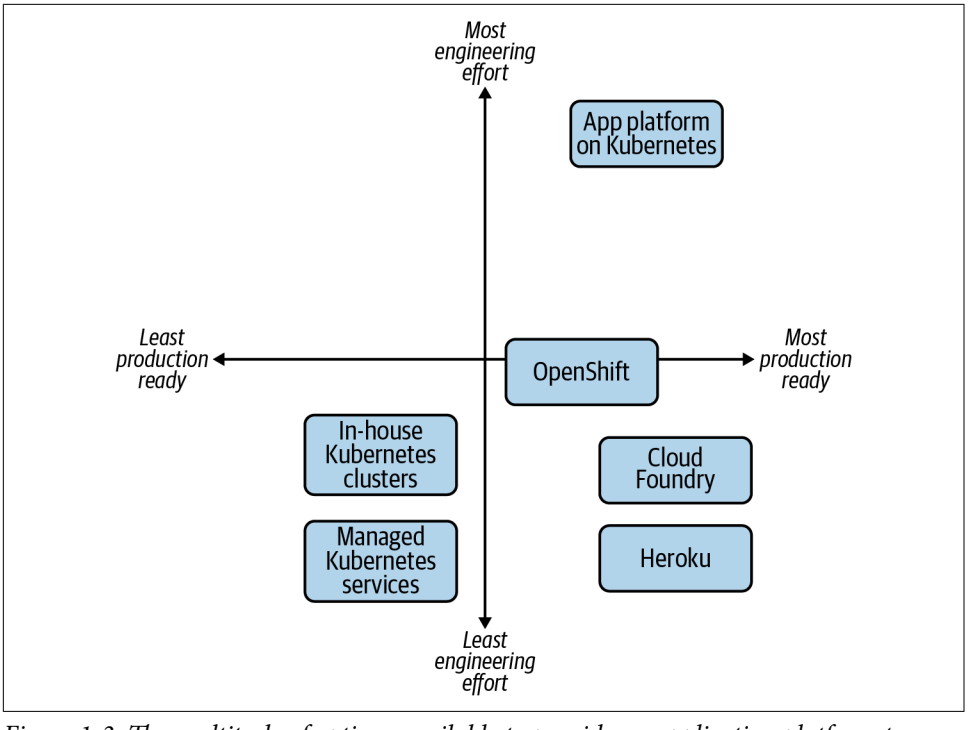

Figure 1-3. The multitude of options available to provide an application platform to developers.

In the bottom right, we have the more established platforms, ones that provide an end-to-end developer experience out of the box. Cloud Foundry is a great example of a project that solves many of the application platform concerns. Running software in Cloud Foundry is more about ensuring the software fits within its opinions. OpenShift, on the other hand, which for most is far more production-ready than just Kubernetes, has more decision points and considerations for how you set it up. Is this flexibility a benefit or a nuisance? That's a key consideration for you.

Lastly, in the top right, we have building an application platform on top of Kubernetes. Relative to the others, this unquestionably requires the most engineering effort, at least from a platform perspective. However, taking advantage of Kubernetes extensibility means you can create something that lines up with your developer, infrastructure, and business needs.

Aligning Your Organizational Needs

What's missing from the graph in Figure 1-3 is a third dimension, a z-axis that demonstrates how aligned the approach is with your requirements. Let's examine another visual representation. Figure 1-4 maps out how this might look when considering platform alignment with organizational needs.

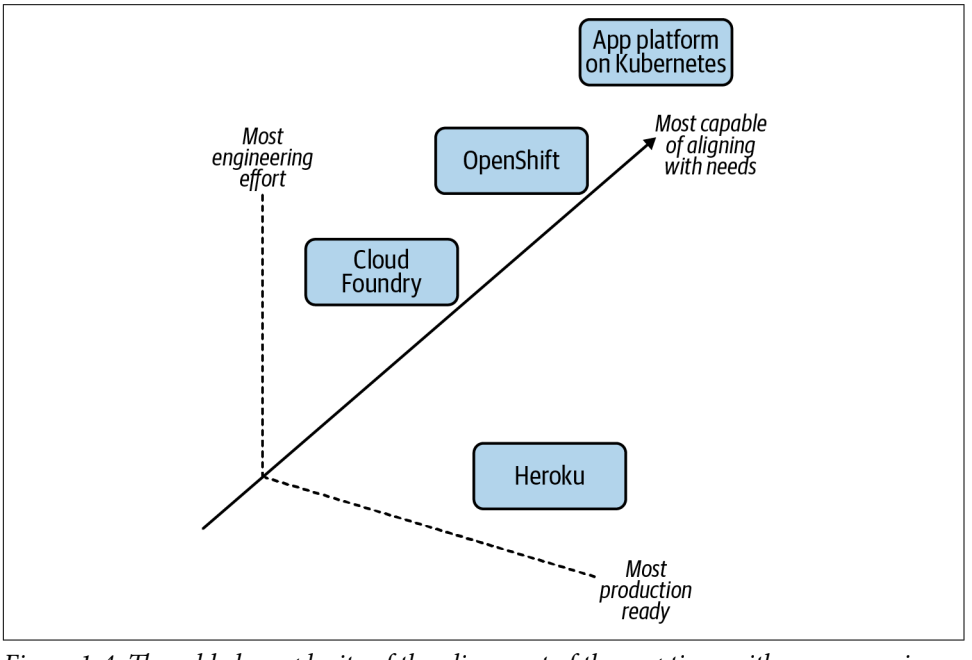

Figure 1-4. The added complexity of the alignment of these options with your organizational needs, the z-axis.

In terms of requirements, features, and behaviors you'd expect out of a platform, building a platform is almost always going to be the most aligned. Or at least the most capable of aligning. This is because you can build anything! If you wanted to reimplement Heroku in-house, on top of Kubernetes, with minor adjustments to its capabilities, it is technically possible. However, the cost/reward should be weighed out with the other axes (x and y). Let's make this exercise more concrete by considering the following needs in a next-generation platform:

- Regulations require you to run mostly on-premise
- Need to support your baremetal fleet along with your vSphere-enabled datacenter
- Want to support growing demand for developers to package applications in containers

- Need ways to build self-service API mechanisms that move you away from "ticket-based" infrastructure provisioning
- Want to ensure APIs you're building atop of are vendor agnostic and not going to cause lock-in because it has cost you millions in the past to migrate off these types of systems
- Are open to paying enterprise support for a variety of products in the stack, but unwilling to commit to models where the entire stack is licensed per node, core, or application instance

We must understand our engineering maturity, appetite for building and empowering teams, and available resources to qualify whether building an application platform is a sensible undertaking.

Summarizing Application Platforms

Admittedly, what constitutes an application platform remains fairly gray. We've focused on a variety of platforms that we believe bring an experience to teams far beyond just workload orchestration. We have also articulated that Kubernetes can be customized and extended to achieve similar outcomes. By advancing our thinking beyond "How do I get a Kubernetes" into concerns such as "What is the current developer workflow, pain points, and desires?" platform and infrastructure teams will be more successful with what they build. With a focus on the latter, we'd argue, you are far more likely to chart a proper path to production and achieve nontrivial adoption. At the end of the day, we want to meet infrastructure, security, and developer requirements to ensure our customers—typically developers—are provided a solution that meets their needs. Often we do not want to simply provide a "powerful" engine that every developer must build their own platform atop of, as jokingly depicted in Figure 1-5.

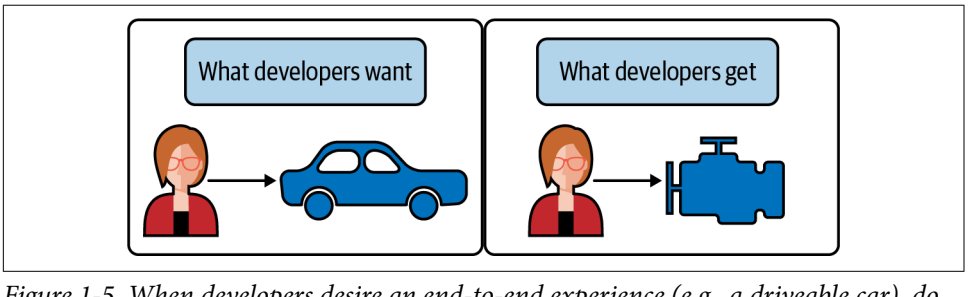

Figure 1-5. When developers desire an end-to-end experience (e.g., a driveable car), do not expect an engine without a frame, wheels, and more to suffice.

Building Application Platforms on Kubernetes

Now we've identified Kubernetes as one piece of the puzzle in our path to production. With this, it would be reasonable to wonder "Isn't Kubernetes just missing stuff then?" The Unix philosophy's principle of "make each program do one thing well" is a compelling aspiration for the Kubernetes project. We believe its best features are largely the ones it does not have! Especially after being burned with one-size-fits-all platforms that try to solve the world's problems for you. Kubernetes has brilliantly focused on being a great orchestrator while defining clear interfaces for how it can be built on top of. This can be likened to the foundation of a home.

A good foundation should be structurally sound, able to be built on top of, and provide appropriate interfaces for routing utilities to the home. While important, a foundation alone is rarely a habitable place for our applications to live. Typically, we need some form of home to exist on top of the foundation. Before discussing *building* on top of a foundation such as Kubernetes, let's consider a pre-furnished apartment as shown in Figure 1-6.

Figure 1-6. An apartment that is move-in ready. Similar to platform as a service options like Heroku. Illustration by Jessica Appelbaum.

This option, similar to our examples such as Heroku, is habitable with no additional work. There are certainly opportunities to customize the experience inside; however, many concerns are solved for us. As long as we are comfortable with the price of rent and are willing to conform to the nonnegotiable opinions within, we can be successful on day one.

Circling back to Kubernetes, which we have likened to a foundation, we can now look to build that habitable home on top of it, as depicted in Figure 1-7.

Figure 1-7. Building a house. Similar to establishing an application platform, which Kubernetes is foundational to. Illustration by Jessica Appelbaum.

At the cost of planning, engineering, and maintaining, we can build remarkable platforms to run workloads throughout organizations. This means we're in complete control of every element in the output. The house can and should be tailored to the needs of the future tenants (our applications). Let's now break down the various layers and considerations that make this possible.

Starting from the Bottom

First we must start at the bottom, which includes the technology Kubernetes expects to run. This is commonly a datacenter or cloud provider, which offers compute, storage, and networking. Once established, Kubernetes can be bootstrapped on top. Within minutes you can have clusters living atop the underlying infrastructure. There are several means of bootstrapping Kubernetes, and we'll cover them in depth in Chapter 2.

From the point of Kubernetes clusters existing, we next need to look at a conceptual flow to determine what we should build on top. The key junctures are represented in Figure 1-8.

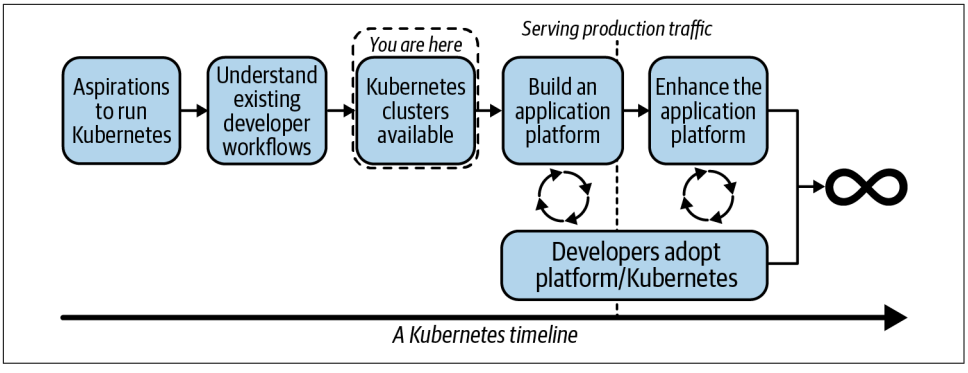

Figure 1-8. A flow our teams may go through in their path to production with Kubernetes.

From the point of Kubernetes existing, you can expect to quickly be receiving questions such as:

- "How do I ensure workload-to-workload traffic is fully encrypted?"
- "How do I ensure egress traffic goes through a gateway guaranteeing a consistent source CIDR?"
- "How do I provide self-service tracing and dashboards to applications?"
- "How do I let developers onboard without being concerned about them becoming Kubernetes experts?"

This list can be endless. It is often incumbent on us to determine which requirements to solve at a platform level and which to solve at an application level. The key here is to deeply understand exiting workflows to ensure what we build lines up with current expectations. If we cannot meet that feature set, what impact will it have on the development teams? Next we can start the building of a platform on top of Kubernetes. In doing so, it is key we stay paired with development teams willing to onboard early and understand the experience to make informed decisions based on quick feedback. After reaching production, this flow should not stop. Platform teams should not expect what is delivered to be a static environment that developers will use for decades. In order to be successful, we must constantly be in tune with our development groups to understand where there are issues or potential missing features that could increase development velocity. A good place to start is considering what level of interaction with Kubernetes we should expect from our developers. This is the idea of how much, or how little, we should abstract.

The Abstraction Spectrum

In the past, we've heard posturing like, "If your application developers know they're using Kubernetes, you've failed!" This can be a decent way to look at interaction with Kubernetes, especially if you're building products or services where the underlying orchestration technology is meaningless to the end user. Perhaps you're building a database management system (DBMS) that supports multiple database technologies. Whether shards or instances of a database run via Kubernetes, Bosh, or Mesos probably doesn't matter to your developers! However, taking this philosophy wholesale from a tweet into your team's success criteria is a dangerous thing to do. As we layer pieces on top of Kubernetes and build platform services to better serve our customers, we'll be faced with many points of decision to determine what appropriate abstractions looks like. Figure 1-9 provides a visualization of this spectrum.

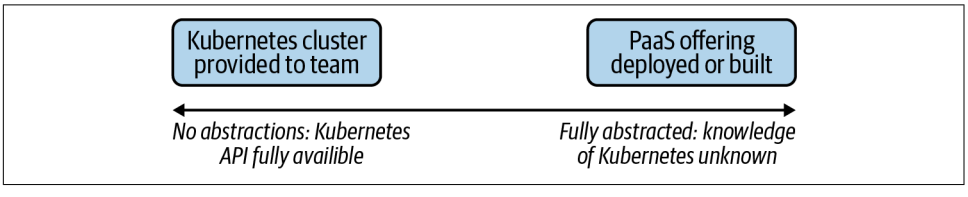

Figure 1-9. The various ends of the spectrum. Starting with giving each team its own Kubernetes cluster to entirely abstracting Kubernetes from your users, via a platform as a service (PaaS) offering.

This can be a question that keeps platform teams up at night. There's a lot of merit in providing abstractions. Projects like Cloud Foundry provide a fully baked developer experience—an example being that in the context of a single `cf push` we can take an application, build it, deploy it, and have it serving production traffic. With this goal and experience as a primary focus, as Cloud Foundry furthers its support for running on top of Kubernetes, we expect to see this transition as more of an implementation detail than a change in feature set. Another pattern we see is the desire to offer more than Kubernetes at a company, but not make developers explicitly choose between technologies. For example, some companies have a Mesos footprint alongside a Kubernetes footprint. They then build an abstraction enabling transparent selection of where workloads land without putting that onus on application developers. It also prevents them from technology lock-in. A trade-off to this approach includes building abstractions on top of two systems that operate differently. This requires significant engineering effort and maturity. Additionally, while developers are eased of the burden around knowing how to interact with Kubernetes or Mesos, they instead need to understand how to use an abstracted company-specific system. In the modern era of open source, developers from all over the stack are less enthused about learning systems that don't translate between organizations. Lastly, a pitfall we've seen is an obsession with abstraction causing an inability to expose key features of Kubernetes.

Over time this can become a cat-and-mouse game of trying to keep up with the project and potentially making your abstraction as complicated as the system it's abstracting.

On the other end of the spectrum are platform groups that wish to offer self-service clusters to development teams. This can also be a great model. It does put the responsibility of Kubernetes maturity on the development teams. Do they understand how Deployments, ReplicaSets, Pods, Services, and Ingress APIs work? Do they have a sense for setting millicpus and how overcommit of resources works? Do they know how to ensure that workloads configured with more than one replica are always scheduled on different nodes? If yes, this is a perfect opportunity to avoid overengineering an application platform and instead let application teams take it from the Kubernetes layer up.

This model of development teams owning their own clusters is a little less common. Even with a team of humans that have a Kubernetes background, it's unlikely that they want to take time away from shipping features to determine how to manage the life cycle of their Kubernetes cluster when it comes time to upgrade. There's so much power in all the knobs Kubernetes exposes, but for many development teams, expecting them to become Kubernetes experts on top of shipping software is unrealistic. As you'll find in the coming chapters, abstraction does not have to be a binary decision. At a variety of points we'll be able to make informed decisions on where abstractions make sense. We'll be determining where we can provide developers the right amount of flexibility while still streamlining their ability to get things done.

Determining Platform Services

When building on top of Kubernetes, a key determination is what features should be built into the platform relative to solved at the application level. Generally this is something that should be evaluated at a case-by-case basis. For example, let's assume every Java microservice implements a library that facilitates mutual TLS (mTLS) between services. This provides applications a construct for identity of workloads and encryption of data over the network. As a platform team, we need to deeply understand this usage to determine whether it is something we should offer or implement at a platform level. Many teams look to solve this by potentially implementing a technology called a service mesh into the cluster. An exercise in trade-offs would reveal the following considerations.

Pros to introducing a service mesh:

- Java apps no longer need to bundle libraries to facilitate mTLS.
- Non-Java applications can take part in the same mTLS/encryption system.
- Lessened complexity for application teams to solve for.

Cons to introducing a service mesh:

- Running a service mesh is not a trivial task. It is another distributed system with operational complexity.
- Service meshes often introduce features far beyond identity and encryption.
- The mesh's identity API might not integrate with the same backend system as used by the existing applications.

Weighing these pros and cons, we can come to the conclusion as to whether solving this problem at a platform level is worth the effort. The key is we don't need to, and should not strive to, solve every application concern in our new platform. This is another balancing act to consider as you proceed through the many chapters in this book. Several recommendations, best practices, and guidance will be shared, but like anything, you should assess each based on the priorities of your business needs.

The Building Blocks

Let's wrap up this chapter by concretely identifying key building blocks you will have available as you build a platform. This includes everything from the foundational components to optional platform services you may wish to implement.

The components in Figure 1-10 have differing importance to differing audiences.

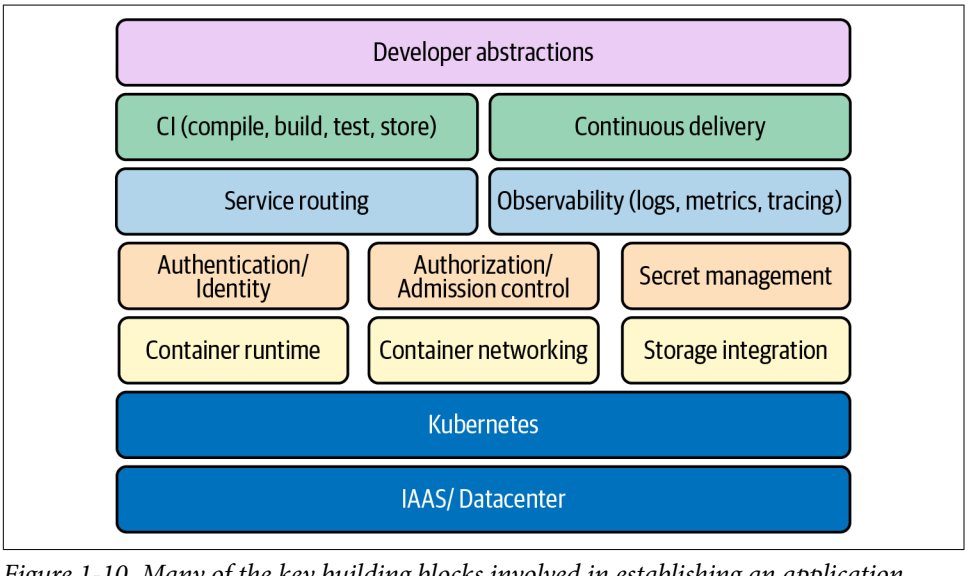

Figure 1-10. Many of the key building blocks involved in establishing an application platform.

Some components such as container networking and container runtime are required for every cluster, considering that a Kubernetes cluster that can't run workloads or allow them to communicate would not be very successful. You are likely to find some components to have variance in whether they should be implemented at all. For example, secret management might not be a platform service you intend to implement if applications already get their secrets from an external secret management solution.

Some areas, such as security, are clearly missing from Figure 1-10. This is because security is not a feature but more so a result of how you implement everything from the IAAS layer up. Let's explore these key areas at a high level, with the understanding that we'll dive much deeper into them throughout this book.

IAAS/datacenter and Kubernetes

IAAS/datacenter and Kubernetes form the foundational layer we have called out many times in this chapter. We don't mean to trivialize this layer because its stability will directly correlate to that of our platform. However, in modern environments, we spend much less time determining the architecture of our racks to support Kubernetes and a lot more time deciding between a variety of deployment options and topologies. Essentially we need to assess how we are going to provision and make available Kubernetes clusters.

Container runtime

The container runtime will faciliate the life cycle management of our workloads on each host. This is commonly implemented using a technology that can manage containers, such as CRI-O, containerd, and Docker. The ability to choose between these different implementations is thanks to the Container Runtime Interface (CRI). Along with these common examples, there are specialized runtimes that support unique requirements, such as the desire to run a workload in a micro-vm.

Container networking

Our choice of container networking will commonly address IP address management (IPAM) of workloads and routing protocols to facilitate communication. Common technology choices include Calico or Cilium, which is thanks to the Container Networking Interface (CNI). By plugging a container networking technology into the cluster, the kubelet can request IP addresses for the workloads it starts. Some plug-ins go as far as implementing service abstractions on top of the Pod network.

Storage integration

Storage integration covers what we do when the on-host disk storage just won't cut it. In modern Kubernetes, more and more organizations are shipping stateful workloads to their clusters. These workloads require some degree of certainty that the state will be resilient to application failure or rescheduling events. Storage can be supplied by common systems such as vSAN, EBS, Ceph, and many more. The ability to choose between various backends is facilitated by the Container Storage Interface (CSI). Similar to CNI and CRI, we are able to deploy a plug-in to our cluster that understands how to satisfy the storage needs requested by the application.

Service routing

Service routing is the facilitation of traffic to and from the workloads we run in Kubernetes. Kubernetes offers a Service API, but this is typically a stepping stone for support of more feature-rich routing capabilities. Service routing builds on container networking and creates higher-level features such as layer 7 routing, traffic patterns, and much more. Many times these are implemented using a technology called an Ingress controller. At the deeper side of service routing comes a variety of service meshes. This technology is fully featured with mechanisms such as service-to-service mTLS, observability, and support for applications mechanisms such as circuit breaking.

Secret management

Secret management covers the management and distribution of sensitive data needed by workloads. Kubernetes offers a Secrets API where sensitive data can be interacted with. However, out of the box, many clusters don't have robust enough secret management and encryption capabilities demanded by several enterprises. This is largely a conversation around defense in depth. At a simple level, we can ensure data is encrypted before it is stored (encryption at rest). At a more advanced level, we can provide integration with various technologies focused on secret management, such as Vault or Cyberark.

Identity

Identity covers the authentication of humans and workloads. A common initial ask of cluster administrators is how to authenticate users against a system such as LDAP or a cloud provider's IAM system. Beyond humans, workloads may wish to identify themselves to support zero-trust networking models where impersonation of workloads is far more challenging. This can be facilitated by integrating an identity provider and using mechanisms such as mTLS to verify a workload.

Authorization/admission control

Authorization is the next step after we can verify the identity of a human or workload. When users or workloads interact with the API server, how do we grant or deny their access to resources? Kubernetes offers an RBAC feature with resource/verb-level controls, but what about custom logic specific to authorization inside our organization? Admission control is where we can take this a step further by building out validation logic that can be as simple as looking over a static list of rules to dynamically calling other systems to determine the correct authorization response.

Software supply chain

The software supply chain covers the entire life cycle of getting software in source code to runtime. This involves the common concerns around continuous integration (CI) and continuous delivery (CD). Many times, developers' primary interaction point is the pipelines they establish in these systems. Getting the CI/CD systems working well with Kubernetes can be paramount to your platform's success. Beyond CI/CD are concerns around the storage of artifacts, their safety from a vulnerability standpoint, and ensuring integrity of images that will be run in your cluster.

Observability

Observability is the umbrella term for all things that help us understand what's happening with our clusters. This includes at the system and application layers. Typically, we think of observability to cover three key areas. These are logs, metrics, and tracing. Logging typically involves forwarding log data from workloads on the host to a target backend system. From this system we can aggregate and analyze logs in a consumable way. Metrics involves capturing data that represents some state at a point in time. We often aggregate, or scrape, this data into some system for analysis. Tracing has largely grown in popularity out of the need to understand the interactions between the various services that make up our application stack. As trace data is collected, it can be brought up to an aggregate system where the life of a request or response is shown via some form of context or correlation ID.

Developer abstractions

Developer abstractions are the tools and platform services we put in place to make developers successful in our platform. As discussed earlier, abstraction approaches live on a spectrum. Some organizations will choose to make the usage of Kubernetes completely transparent to the development teams. Other shops will choose to expose many of the powerful knobs Kubernetes offers and give significant flexibility to every developer. Solutions also tend to focus on the developer onboarding experience, ensuring they can be given access and secure control of an environment they can utilize in the platform.

Summary

In this chapter, we have explored ideas spanning Kubernetes, application platforms, and even building application platforms on Kubernetes. Hopefully this has gotten you thinking about the variety of areas you can jump into in order to better understand how to build on top of this great workload orchestrator. For the remainder of the book we are going to dive into these key areas and provide insight, anecdotes, and recommendations that will further build your perspective on platform building. Let's jump in and start down this path to production!

CHAPTER 2
Deployment Models

The first step to using Kubernetes in production is obvious: make Kubernetes exist. This includes installing systems to provision Kubernetes clusters and to manage future upgrades. Being that Kubernetes is a distributed software system, deploying Kubernetes largely boils down to a software installation exercise. The important difference compared with most other software installs is that Kubernetes is intrinsically tied to the infrastructure. As such, the software installation and the infrastructure it's being installed on need to be simultaneously solved for.

In this chapter we will first address preliminary questions around deploying Kubernetes clusters and how much you should leverage managed services and existing products or projects. For those that heavily leverage existing services, products, and projects, most of this chapter may not be of interest because about 90% of the content in this chapter covers how to approach custom automation. This chapter can still be of interest if you are evaluating tools for deploying Kubernetes so that you can reason about the different approaches available. For those in the uncommon position of having to build custom automation for deploying Kubernetes, we will address overarching architectural concerns, including special considerations for etcd as well as how to manage the various clusters under management. We will also look at useful patterns for managing the various software installations as well as the infrastructure dependencies and will break down the various cluster components and demystify how they fit together. We'll also look at ways to manage the add-ons you install to the base Kubernetes cluster as well as strategies for upgrading Kubernetes and the add-on components that make up your application platform.

Managed Service Versus Roll Your Own

Before we get further into the topic of deployment models for Kubernetes, we should address the idea of whether you should even *have* a full deployment model for Kubernetes. Cloud providers offer managed Kubernetes services that mostly alleviate the deployment concerns. You should still develop reliable, declarative systems for provisioning these managed Kubernetes clusters, but it may be advantageous to abstract away most of the details of *how* the cluster is brought up.

Managed Services

The case for using managed Kubernetes services boils down to savings in engineering effort. There is considerable technical design and implementation in properly managing the deployment and life cycle of Kubernetes. And remember, Kubernetes is just one component of your application platform—the container orchestrator.

In essence, with a managed service you get a Kubernetes control plane that you can attach worker nodes to at will. The obligation to scale, ensure availability, and manage the control plane is alleviated. These are each significant concerns. Furthermore, if you already use a cloud provider's existing services you get a leg up. For example, if you are in Amazon Web Services (AWS) and already use Fargate for serverless compute, Identity and Access Management (IAM) for role-based access control, and CloudWatch for observability, you can leverage these with their Elastic Kubernetes Service (EKS) and solve for several concerns in your app platform.

It is not unlike using a managed database service. If your core concern is an application that serves your business needs, and that app requires a relational database, but you cannot justify having a dedicated database admin on staff, paying a cloud provider to supply you with a database can be a huge boost. You can get up and running faster. The managed service provider will manage availability, take backups, and perform upgrades on your behalf. In many cases this is a clear benefit. But, as always, there is a trade-off.

Roll Your Own

The savings available in using a managed Kubernetes service come with a price tag. You pay with a lack of flexibility and freedom. Part of this is the threat of vendor lock-in. The managed services are generally offered by cloud infrastructure providers. If you invest heavily in using a particular vendor for your infrastructure, it is highly likely that you will design systems and leverage services that will not be vendor neutral. The concern is that if they raise their prices or let their service quality slip in the future, you may find yourself painted into a corner. Those experts you paid to handle concerns you didn't have time for may now wield dangerous power over your destiny.

Of course, you can diversify by using managed services from multiple providers, but there will be deltas between the way they expose features of Kubernetes, and which features are exposed could become an awkward inconsistency to overcome.

For this reason, you may prefer to roll your own Kubernetes. There is a vast array of knobs and levers to adjust on Kubernetes. This configurability makes it wonderfully flexible and powerful. If you invest in understanding and managing Kubernetes itself, the app platform world is your oyster. There will be no feature you cannot implement, no requirement you cannot meet. And you will be able to implement that seamlessly across infrastructure providers, whether they be public cloud providers, or your own servers in a private datacenter. Once the different infrastructure inconsistencies are accounted for, the Kubernetes features that are exposed in your platform will be consistent. And the developers that use your platform will not care—and may not even know—who is providing the underlying infrastructure.

Just keep in mind that developers will care only about the features of the platform, not the underlying infrastructure or who provides it. If you are in control of the features available, and the features you deliver are consistent across infrastructure providers, you have the freedom to deliver a superior experience to your devs. You will have control of the Kubernetes version you use. You will have access to all the flags and features of the control plane components. You will have access to the underlying machines and the software that is installed on them as well as the static Pod manifests that are written to disk there. You will have a powerful and dangerous tool to use in the effort to win over your developers. But never ignore the obligation you have to learn the tool well. A failure to do so risks injuring yourself and others with it.

Making the Decision

The path to glory is rarely clear when you begin the journey. If you are deciding between a managed Kubernetes service or rolling your own clusters, you are much closer to the beginning of your journey with Kubernetes than the glorious final conclusion. And the decision of managed service versus roll your own is fundamental enough that it will have long-lasting implications for your business. So here are some guiding principles to aid the process.

You should lean toward a managed service if:

- The idea of understanding Kubernetes sounds terribly arduous
- The responsibility for managing a distributed software system that is critical to the success of your business sounds dangerous
- The inconveniences of restrictions imposed by vendor-provided features seem manageable

- You have faith in your managed service vendor to respond to your needs and be a good business partner

You should lean toward rolling your own Kubernetes if:

- The vendor-imposed restrictions make you uneasy
- You have little or no faith in the corporate behemoths that provide cloud compute infrastructure
- You are excited by the power of the platform you can build around Kubernetes
- You relish the opportunity to leverage this amazing container orchestrator to provide a delightful experience to your devs

If you decide to use a managed service, consider skipping most of the remainder of this chapter. "Add-ons" on page 50 and "Triggering Mechanisms" on page 60 are still applicable to your use case, but the other sections in this chapter will not apply. If, on the other hand, you are looking to manage your own clusters, read on! Next we'll dig more into the deployment models and tools you should consider.

Automation

If you are to undertake designing a deployment model for your Kubernetes clusters, the topic of automation is of the utmost importance. Any deployment model will need to keep this as a guiding principle. Removing human toil is critical to reduce cost and improve stability. Humans are costly. Paying the salary for engineers to execute routine, tedious operations is money not spent on innovation. Furthermore, humans are unreliable. They make mistakes. Just one error in a series of steps may introduce instability or even prevent the system from working at all. The upfront engineering investment to automate deployments using software systems will pay dividends in saved toil and troubleshooting in the future.

If you decide to manage your own cluster life cycle, you must formulate your strategy for doing this. You have a choice between using a prebuilt Kubernetes installer or developing your own custom automation from the ground up. This decision has parallels with the decision between managed services versus roll your own. One path gives you great power, control, and flexibility but at the cost of engineering effort.

Prebuilt Installer

There are now countless open source and enterprise-supported Kubernetes installers available. Case in point: there are currently 180 Kubernetes Certified Service Providers listed on the CNCF's website. Some you will need to pay money for and will be accompanied by experienced field engineers to help get you up and running as well as support staff you can call on in times of need. Others will require research and

experimentation to understand and use. Some installers—usually the ones you pay money for—will get you from zero to Kubernetes with the push of a button. If you fit the prescriptions provided and options available, and your budget can accommodate the expense, this installer method could be a great fit. At the time of this writing, using prebuilt installers is the approach we see most commonly in the field.

Custom Automation

Some amount of custom automation is commonly required even if using a prebuilt installer. This is usually in the form of integration with a team's existing systems. However, in this section we're talking about developing a custom Kubernetes installer.

If you are beginning your journey with Kubernetes or changing direction with your Kubernetes strategy, the homegrown automation route is likely your choice only if all of the following apply:

- You have more than just one or two engineers to devote to the effort
- You have engineers on staff with deep Kubernetes experience
- You have specialized requirements that no managed service or prebuilt installer satisfies well

Most of the remainder of this chapter is for you if one of the following applies:

- You fit the use case for building custom automation
- You are evaluating installers and want to gain a deeper insight into what good patterns look like

This brings us to details of building custom automation to install and manage Kubernetes clusters. Underpinning all these concerns should be a clear understanding of your platform requirements. These should be driven primarily by the requirements of your app development teams, particularly those that will be the earliest adopters. Do not fall into the trap of building a platform in a vacuum without close collaboration with the consumers of the platform. Make early prerelease versions of your platform available to dev teams for testing. Cultivate a productive feedback loop for fixing bugs and adding features. The successful adoption of your platform depends upon this.

Next we will cover architecture concerns that should be considered before any implementation begins. This includes deployment models for etcd, separating deployment environments into tiers, tackling challenges with managing large numbers of clusters, and what types of node pools you might use to host your workloads. After that, we'll get into Kubernetes installation details, first for the infrastructure dependencies, then, the software that is installed on the clusters' virtual or physical machines, and finally for the containerized components that constitute the control plane of a Kubernetes cluster.

Architecture and Topology

This section covers the architectural decisions that have broad implications for the systems you use to provision and manage your Kubernetes clusters. They include the deployment model for etcd and the unique considerations you must take into account for that component of the platform. Among these topics is how you organize the various clusters under management into tiers based on the service level objectives (SLOs) for them. We will also look at the concept of node pools and how they can be used for different purposes within a given cluster. And, lastly, we will address the methods you can use for federated management of your clusters and the software you deploy to them.

etcd Deployment Models

As the database for the objects in a Kubernetes cluster, etcd deserves special consideration. etcd is a distributed data store that uses a consensus algorithm to maintain a copy of the your cluster's state on multiple machines. This introduces network considerations for the nodes in an etcd cluster so they can reliably maintain that consensus over their network connections. It has unique network latency requirements that we need to design for when considering network topology. We'll cover that topic in this section and also look at the two primary architectural choices to make in the deployment model for etcd: dedicated versus colocated and whether to run in a container or install directly on the host.

Network considerations

The default settings in etcd are designed for the latency in a single datacenter. If you distribute etcd across multiple datacenters, you should test the average round-trip between members and tune the heartbeat interval and election timeout for etcd if need be. We strongly discourage the use of etcd clusters distributed across different regions. If using multiple datacenters for improved availability, they should at least be in close proximity within a region.

Dedicated versus colocated

A very common question we get about how to deploy is whether to give etcd its own dedicated machines or to colocate them on the control plane machines with the API server, scheduler, controller manager, etc. The first thing to consider is the size of clusters you will be managing, i.e., the number of worker nodes you will run per cluster. The trade-offs around cluster sizes will be discussed later in the chapter. Where you land on that subject will largely inform whether you dedicate machines to etcd. Obviously etcd is crucial. If etcd performance is compromised, your ability to control the resources in your cluster will be compromised. As long as your workloads don't

have dependencies on the Kubernetes API, they should not suffer, but keeping your control plane healthy is still very important.

If you are driving a car down the street and the steering wheel stops working, it is little comfort that the car is still driving down the road. In fact, it may be terribly dangerous. For this reason, if you are going to be placing the read and write demands on etcd that come with larger clusters, it is wise to dedicate machines to them to eliminate resource contention with other control plane components. In this context, a "large" cluster is dependent upon the size of the control plane machines in use but should be at least a topic of consideration with anything above 50 worker nodes. If planning for clusters with over 200 workers, it's best to just plan for dedicated etcd clusters. If you do plan smaller clusters, save yourself the management overhead and infrastructure costs—go with colocated etcd. Kubeadm is a popular Kubernetes bootstrapping tool that you will likely be using; it supports this model and will take care of the associated concerns.

Containerized versus on host

The next common question revolves around whether to install etcd on the machine or to run it in a container. Let's tackle the easy answer first: if you're running etcd in a colocated manner, run it in a container. When leveraging kubeadm for Kubernetes bootstrapping, this configuration is supported and well tested. It is your best option. If, on the other hand, you opt for running etcd on dedicated machines your options are as follows: you can install etcd on the host, which gives you the opportunity to bake it into machine images and eliminate the additional concerns of having a container runtime on the host. Alternatively, if you run in a container, the most useful pattern is to install a container runtime and kubelet on the machines and use a static manifest to spin up etcd. This has the advantage of following the same patterns and install methods as the other control plane components. Using repeated patterns in complex systems is useful, but this question is largely a question of preference.

Cluster Tiers

Organizing your clusters according to tiers is an almost universal pattern we see in the field. These tiers often include testing, development, staging, and production. Some teams refer to these as different "environments." However, this is a broad term that can have different meanings and implications. We will use the term *tier* here to specifically address the different types of clusters. In particular, we're talking about the SLOs and SLAs that may be associated with the cluster, as well as the purpose for the cluster, and where the cluster sits in the path to production for an application, if at all. What exactly these tiers will look like for different organizations varies, but there are common themes and we will describe what each of these four tiers commonly mean. Use the same cluster deployment and life cycle management systems across all